

What Stops a Small Language Model From Driving a Database Agent

Cevheri Bozođlan Yusuf Gündođdu Abdullah Kaya
Koray řirin

Independent researchers, and contributors to the software evaluated
github.com/libredb/libredb-studio

September 18, 2026

Abstract

Small open-weight language models are assumed to fail at agentic database work because they lack the reasoning capacity for it. We test that against a production system. Over eleven days we drove the agent mode of an open-source SQL client with 39 open-weight models served locally and one hosted control, across six task surfaces: 8,199 runs, 110,711 ledger events, 14,008 refused tool calls. Of the 2,100 model-attributed agent-mode losses, 1,590, or 75.7%, came from runs that had invoked at least one tool. That majority is what survives resampling models rather than runs: it holds in 99.7% of clustered resamples and in 15 of the 22 models with at least twenty losses. Within it, transport, a run that used the tools and never got a deliverable through, is the largest class at 36.2% and capability, a run that invoked no tool at all, the smallest at 17.3%; we report that ordering as a property of this corpus rather than a general finding, since clustered by model it holds in only 74.5% of resamples. Transport failures decompose into a few mechanical argument shapes. Production ledgers record refusal codes and never the model’s arguments, so these were invisible for ten days; capturing them exposed five server defects, one of which demanded a field on one tool, forbade it on the sibling that composed it, then failed the run for its absence. Five server changes, touching no model, prompt or sampling setting, moved six models by 6 to 21 cells out of 30. We also report a confound we believe affects published local-model benchmarks, ours included: with no context cap, one 7.1 GB model was admitted at its full 262,144-token window and held 51 GB on a 64 GB machine, producing runs indistinguishable in any ordinary log from a model timing out. The corpus, the scorer and a verifier that regenerates every figure are released.

1 Introduction

The question we set out to answer was operational. LibreDB Studio ships an agent mode that plans and executes read-only database work through a fixed tool contract. Its users run it against their own databases, and a substantial fraction want to run it against a model on their own hardware rather than a hosted API, because sending schema inventories and query text to a third party is exactly what their environment forbids. Which locally hosted models can actually drive that agent, and what has to change so that more of them can?

The prevailing framing, in vendor documentation and practitioner writing alike, is a capability threshold: below some parameter count a model cannot sustain a tool-calling loop. That framing makes a specific, checkable prediction. Failures should look like models that do not act. What

we observe is close to the opposite. Most failures are models that act, establish something true, and then lose it at the boundary between the model and the server.

We are not the first to report that agent harnesses are implicated in what looks like model failure, and we say so at the outset. Section 2 places this work against results that predate it. Our contribution is a field record at production scale, with the ledger, the extractor and the argument captures released, and with our own methodological choices and confounds reported rather than smoothed.

This paper contributes:

1. A four-class failure taxonomy for agentic runs, capability, transport, clock and verification, computable over a structured run ledger, together with its measured distribution across 40 models and six surfaces, and an explicit sensitivity analysis of the one classification decision that changes the headline.
2. A characterisation of the transport class from captured tool-call arguments, showing that it decomposes into a small number of mechanical shapes in which the model was substantively right.
3. Five server-side interventions with before and after readings on the same hardware, prompts and tasks, including an honest separation of what did and did not move the numbers.
4. Two measurement confounds, unbounded serving context and swap pressure, that produce results indistinguishable from model failure and that we believe affect published local-model evaluations.

2 Related work

Feedback quality as the binding constraint. Olausson et al. [1] showed that self-repair in code generation is bottlenecked by the quality of the feedback rather than by the repairer’s ability to act on it. Closest to the present work, Gumaan [2] measured the change in log-probability of re-emitting an action after its failure is recorded in the transcript, found this quantity negative for every instruction-tuned model tested, six checkpoints from 135M to 1.7B across four families, and reported the probability of repeating a failed call rising from 0.06 to 0.54. Counterfactuals there attribute 83% of the damage to the failed call’s surface form in the context rather than to the semantics of marking it failed, and show that replacing the verbatim call with a runtime-generated description of the failure removes 76% of the repetition. That work establishes both the phenomenon and a direction of repair, and it predates our measurements. Wang et al. [3] quantified feedback’s contribution in multi-turn interaction and found that better single-turn performance does not imply better multi-turn performance. Huang et al. [4] ruled out intrinsic self-correction as the mechanism, and Gou et al. [5] showed that external tool feedback is what drives correction.

We differ in three ways. Our models are an order of magnitude larger, 1.7B to 35B for the tags that name a size, rather than 135M to 1.7B. Our outcome is end-to-end task success under a server-side verifier rather than a log-probability proxy. And our setting is a deployed product against a live database, so the evidence check runs inside the loop: a report is refused by the server during the run rather than scored after it, which is what makes verification a cause of loss here rather than a label applied to a finished trace.

The same failure, repaired by training. Zhang et al. [6] report independently that smaller models after an execution error often fall into repetitive invalid re-inocations, and address it with reinforcement learning on simulated diagnostic feedback. Vuddanti et al. [7] train on failure-injected trajectories. Kadekodi et al. [8] decouple tool selection from argument generation and find argument generation the weaker half, which is exactly where our refusals occur. Our interventions require no gradient updates and help every model at once, which is the practical difference.

Tool-use benchmarks. BFCL [9], ToolLLM [10], API-Bank [11], ToolSandbox [12] and τ -bench [13] vary the model against a harness whose rejection messages are fixed and form part of the benchmark. Holding those messages fixed is what makes scores comparable across models, and it is also what places the harness outside what those five can measure.

The harness as an experimental variable. That axis is not ours to claim. Sigdel and Baral isolate the tool interface directly [14], comparing free-form documentation, JSON Schema, and JSON Schema with structured validation diagnostics under identical tool semantics, and report that the schema conditions reduce interface misuse but not semantic misuse; that pilot runs one open local model and ends at zero task success in every condition. Their ToolMisuseBench [15] makes the interface a declared condition rather than a fixture, with per-task fault plans replayed under a fixed seed across schema drift, rate limits, timeouts, authorization failures and adversarial error rewriting, so the text returned after a failed call is itself a variable; its baselines are deterministic policies rather than language models, and its environments are simulators that need no external service. AgentCheck [16] records an agent’s real tool responses, replays them with exactly one perturbed, and re-runs under a mitigation wrapper against the identical fault, which is a reproduce-and-intervene loop like the one in Section 7, applied at the tool layer rather than to the server’s contract. Xiong et al. [17] hold the agent scaffold fixed and apply fifteen perturbations to the three input sources a tool agent reads, the tool document, the user query and the tool return, and close by recommending that tool error messages be redesigned so that models can correct failures.

What this work adds is the setting and the provenance rather than the axis. The harness defects reported here were not injected. They were live in a shipped validator, recovered from a production ledger, and then repaired in the product, and the effect of repairing them is measured across 40 models rather than a scripted policy or a single pilot model. The change those studies recommend is the change we made and then measured.

The same separation, the opposite ordering. ToolFailBench [18] draws the distinction Table 4 draws, between a run that never called a tool and one that called it and then failed to use the result, and reports the first as much the larger class across nineteen models. Its protocol is single-turn against mock tools that return a controlled value, with the final answer written in a second step and no further turn in which to repair a rejected call, so a call the system did not execute is recorded as a skip rather than answered with a refusal the model could act on. Its own rule counts a tool call emitted as plain text in the answer body and never executed within that skip class; we count the same observable as a transport failure of the surface that did not parse it, and repair it in Section 7. We read the two results as the same argument from opposite ends. Which class a tool-calling failure falls into is a joint property of the model and the harness, and a design that varies only one of them settles the attribution in advance.

Agent benchmarks that attribute failure to capability. AgentBench [19] evaluates across eight environments including a database environment and attributes the gap between open and commercial models to long-term reasoning, decision making and instruction following. It is both the nearest prior database-agent evaluation and the clearest statement of the position our results complicate.

Database-side work. The text-to-SQL lineage, Spider [20] and BIRD [21], evaluates a single emitted statement. SPaC [22] and CoSQL [23] are multi-turn, but the counterpart in the loop is a human, so they cannot exhibit this failure mode. Spider 2.0 [24] is the nearest prior art on the agent axis. D-Bot [25] is the closest sibling to our setting, an agent that reads a live system and writes a report. CodeS [26] makes small models competitive at text-to-SQL by training them for it; our result concerns off-the-shelf models once the harness is repaired. Belcak et al. [27] argue on general grounds that small language models suit agentic work, a position for which this paper supplies field evidence without testing it directly.

3 System under test

LibreDB Studio’s agent is a constrained loop with a verifier, not a general assistant with database tools attached. Three properties matter for reading the results.

Six surfaces. Five run in agent mode, *investigation*, *query-optimization*, *database-assessment*, *operations* and *data-analysis*, and one in planning mode. Each carries its own objective, tool set and verifier. They are not difficulty tiers but different shapes of work. Planning is toolless: the server hands a planning run an empty tool set, so no planning run can invoke a tool. The server still reads the database before a planning run’s first turn, with statements it composes itself; what planning does not do is let the model choose a read. Data analysis must both report and *present* a result. Query optimization must produce a recommendation the user’s editor can apply.

A tool contract with evidence. Every claim in a report must cite an artifact the run actually produced: the correlation id of a completed read, or the fingerprint of a schema snapshot the run captured. Citations are checked against the run’s own ledger and refused when they do not resolve. This is a deliberate anti-hallucination measure and, as Section 5 shows, a substantial source of loss.

A per-model settings layer. The server carries a settings layer that can override sampling, the per-call timeout, reminder limits and several loop behaviours for a named model. Ten of the models in this corpus carry an entry in it, covering 1,013 of the 4,951 model-attributed runs; the apparatus in Section 4.2 is the default that applies to the rest. None of the six models in the intervention table carries an entry: all six ran at the compiled defaults both before and after, so no per-model setting differs between their two readings.

A verifier, not a human judge. Each run ends with a verdict recording *answered* or *unanswered* together with a list naming what was missing. The pass label is produced by the same code path for every run, with no post-hoc scoring, which is what makes the corpus analysable at all. Critically, *status*: *succeeded* is not a pass: a run that reports nothing also exits successfully. In our

corpus 2,204 runs ended `model-stopped` and 1,405 of those were unanswered, so conflating process exit with task success would inflate the apparent pass rate by that margin.

4 Method

4.1 The cell and the lock

The unit of measurement is a *cell*: one model on one surface, run five consecutive times against a fixed objective. Five consecutive passes locks the cell, and a model is complete at 30/30 over six cells. We report the pair rather than a percentage, because the denominator is what makes two readings comparable.

Two bookkeeping rules govern this, and both were learned by violating them. A lock is never taken back: a cell at 5/5 is not re-measured or re-tuned. And a row must be read in one sitting under one configuration. Two models were recorded at 30/30 and were not: their cells were genuinely 5/5, thirty runs and thirty passes all on the ledger, but taken across three days and several settings. Read together in one sitting they returned 22/30 and 23/30. A model ships with *a* reading, never with the union of readings.

4.2 Apparatus

Machine	MacBook Pro (Mac17,6), Apple M5 Max, 18 cores, 64 GB unified memory
Operating system	macOS 26.6
Local inference	Ollama 0.33.0; context length unset for every figure in §5 (model default, up to 262,144 tokens), capped at 32,768 only for the confirmations in §8.1
Runtime	Bun 1.4.2, Next.js 16
Database	embedded SQLite sample, employee/department/salary schema
Turn limit	90,000 ms default; 150,000 ms is the maximum any shipped model carries
Hosted control	<code>gemini-3.5-flash-lite</code>

Table 1: Apparatus. The sample database is deliberately small and fixed: we measure the model’s ability to drive the loop, not the database’s ability to be large, and a fixed schema keeps the objective identical across all 8,199 runs.

4.3 What the ledger records, and what it does not

Every run writes a framed-JSON event stream: run started, driver resolved, context captured, tool invoked, tool completed, tool refused, call declined, call held, guidance issued, model stopped saying, report composed, answer composed, run finished.

Analysis is done from the ledger and never from the sweep logs, and the distinction is not pedantic. A sweep log records what a runner asked for; the ledger records what happened. They have disagreed materially: one chain reported forty-eight completed runs in a minute having run none, and a log header printing `LLM_PROVIDER=ollama` belonged to runs the ledger shows went to a hosted API.

One omission in the ledger is deliberate and became the central methodological finding of this work. A declined call records the tool, the refusal code and the validator’s field paths, and never the model’s arguments, so that model-authored text cannot enter the server’s own audit

vocabulary. That is correct for a production ledger. It also meant that for ten measurement days we could see *that* a required field was absent and never *where the model had put it instead*. Section 6 reports what happened when we added a temporary, environment-gated argument dump.

4.4 Classification, and the decision that matters

We classify each loss by what the ledger shows the run did:

- **clock**: the run ended `model-timeout`, `turn-limit` or `deadline-exceeded`.
- **capability**: the run invoked no tool at all.
- **verification**: the run ended `report-composed` and was still unanswered, that is, a report was filed and rejected.
- **transport**: the run invoked tools, did not run out of time, and never got a deliverable through.

These are applied in that order, and **the order is load-bearing**. Of the 2,100 model-attributed agent-mode losses, 231 both ran out of clock and invoked no tool, so they satisfy the first two definitions simultaneously. We report both assignments in Section 5 and adopt clock-first, for a reason that is empirical rather than stylistic: 225 of those 231 runs also emitted no text at all. A run that produced neither a tool call nor a single character within its turn limit does not look like a model declining to act; it looks like a run that never got going, which is the signature of the memory confound in §8.1. We regard the six runs that did emit text, a median of about 2,000 characters, as genuinely mixed cases.

5 Results

5.1 Corpus

Runs	8,199
Named models	40
open-weight, served locally	39
hosted, proprietary	1
Runs attributable to a named model	4,951
Runs predating the driver-resolved ledger event	3,248
Surfaces	6
Code checkouts the corpus spans	5
Answered	4,983 (60.8%)
Unanswered	3,142 (38.3%)
Runs with no recorded verdict	74 (0.9%)
Refused tool calls	14,008

Table 2: The corpus. Run counts per named model range from 3 to 629, with 35 models at 20 runs or more.

Surface	Answered	n	Rate
planning	466	560	83.2%
investigation	389	550	70.7%
operations	404	612	66.0%
database-assessment	342	614	55.7%
query-optimization	639	1,360	47.0%
data-analysis	468	1,206	38.8%

Table 3: Pass rate by surface, model-attributed runs only. The ordering does not track the intuitive difficulty of the question. It tracks how many schema-constrained objects the model must emit: planning asks for one statement, data analysis asks for a presented answer and a cited report.

5.2 Pass rate by surface

5.3 The failure taxonomy

Class	clock-first (adopted)		capability-first	
	Count	Share	Count	Share
transport	761	36.2%	761	36.2%
clock	542	25.8%	395	18.8%
verification	434	20.7%	434	20.7%
capability	363	17.3%	510	24.3%

Table 4: The 2,100 model-attributed agent-mode losses under both admissible orderings of the two overlapping definitions in §4.4. Under the ordering we adopt, capability is the smallest class; under the alternative it is the second largest and verification becomes the smallest. Transport is the largest class either way, and that is the finding that does not depend on the choice.

Table 4 is the paper’s central result and we state its limits with it. The capability-threshold framing predicts that the capability class should dominate. Under our adopted ordering it is the smallest of four, and more than four losses in five come from runs that engaged the tools. Under the alternative ordering capability rises to 24.3% and the claim weakens to a different one: transport alone still exceeds it, and the majority of losses still come from runs that used the tools, but capability is no longer the smallest class. Readers who prefer the stricter reading should take the second pair of columns and the weaker claim.

Why this table covers agent mode only. Planning is toolless by construction, so every one of its 986 runs invokes zero tools and every one of its 182 losses lands in clock or capability by definition, and none can ever enter transport. Including planning would therefore let an entire surface inflate the two classes our headline compares while being structurally barred from the third. Reporting agent mode alone removes that artefact, and it moves the result in the direction that makes it harder for us, not easier: it raises the class we are arguing for and lowers the one we are arguing against only by removing runs that could never have contradicted us. For completeness, over all 3,142 losses including planning and unattributed runs the adopted ordering gives transport 33.3%, clock 26.5%, verification 20.6% and capability 19.7%.

The transport class is not marginal engagement. Across those 761 runs the median tool-invocation count is 2 and the total is 1,926 invocations against 1,219 further declined calls: these are runs that inspected schemas, ran reads and examined plans, and then failed to file.

5.4 How a cell is scored, and how much the choice is worth

A cell’s history is not one experiment. It spans code changes and configuration changes, so how a scorer searches that history for five consecutive passes decides the answer. The released scorer implements three modes and we report the gap between them, because it is large and because the obvious implementation is the one that overstates.

Pooled scans a cell’s entire run history for any streak of five consecutive passes. It is the obvious implementation. *Session* splits the history into sittings and requires the streak to fall inside one, but allows different cells to come from different sittings. *Row*, which is our protocol, splits the history into sittings, scores each sitting on its own and takes the best, so all six cells must lock inside one sitting.

On this corpus the difference is not marginal. Two models score 6/6 and 30/30 pooled and 23/30 under row. When those two were re-read by hand, six surfaces in one sitting, they returned 24/30 and 21/30, which is what row predicts and not what pooled does. A pooled streak says a model passed five times running under *some* mixture of conditions, which is not a claim a deployment can act on. Every per-model figure in this paper is a row score, and the released scorer reproduces all three so that a reader can see the gap rather than take our word for it.

5.5 Per-model results

5.6 Where the refusals are

6 What transport failures actually look like

The taxonomy says the largest class is work done and delivery failed. This section reports what the model text and arguments show inside it, because the mechanism is specific and repairable.

6.1 The call written into the wrong channel

Of 69 stopping turns whose text we examined, 27 contained a complete tool call for a tool the run held, 18 of them the reporting tool whose absence *is* the missing-report verdict. A representative case, verbatim from the ledger:

```
{"action": "compose_report", "arguments": {"claims": [
  {"claim": "The current query plan uses a full table scan, which can be slow for large tables.",
   "evidence": [{"source": "artifact", "correlationId": "935c1381-d06a-4692-a02a-fd466b7ad62a"}]},
  {"claim": "Adding an index on first_name and last_name will improve performance ...",
   "evidence": [{"source": "artifact", "correlationId": "935c1381-d06a-4692-a02a-fd466b7ad62a"}]}
]}
```

The claims are true, the evidence arrays are populated, and the correlation id is one the run genuinely produced. The model wrote this into the assistant text channel instead of calling with it, and the run ended having done every part of the work except the transport.

None of the 27 parsed as JSON. Twenty-four failed identically, on an array whose last member closes twice. Twenty used the key `name` and seven used `action`. Any recovery path that requires a JSON parse to succeed sees none of them.

Model	Runs	Cells passed (of 30)	Surfaces locked (of 6)
qwen3:4b	73	30/30	6/6
qwen3.6:35b	137	30/30	6/6
qwen3.5:27b	30	30/30	6/6
qwen3-coder:30b	71	30/30	6/6
gpt-oss:20b	272	30/30	6/6
granite4.2:3b	131	28/30	5/6
gemini-3.5-flash-lite	90	28/30	5/6
qwq:32b	50	28/30	4/6
deepseek-r1:8b	383	27/30	5/6
mistral-small3.2:24b	548	25/30	5/6
granite4:3b	85	25/30	5/6
magistral:24b	49	25/30	4/6
glm-4.7-flash:latest	217	25/30	4/6
mistral-nemo:12b	398	24/30	4/6
llama3.1:8b	629	23/30	4/6
granite4.1:3b	70	23/30	4/6
cogito:8b	551	23/30	3/6
ministral-3:3b	119	23/30	2/6
phi4-mini:3.8b	30	22/30	4/6
command-r7b:7b	115	20/30	3/6
lfn2:24b	45	17/30	3/6
hermes3:8b	145	17/30	3/6
glm4:latest	223	17/30	3/6
qwen3.5:2b	60	16/30	2/6
qwen3-vl:8b	81	14/30	2/6
qwen3:1.7b	55	12/30	1/6
deepseek-r1:14b	30	12/30	1/6
mistral-small3.1:24b	35	10/30	2/6
qwen2:7b	33	8/30	1/6
mistral:7b	30	5/30	1/6
qwen3-1.7b	30	0/30	0/6

Table 5: Every model with a complete six-surface reading in the released corpus, scored by the released scorer. A surface is locked only by five consecutive passes, scored in row mode (§5.4) over each model’s whole history. Nine of the 31 lock every surface; one locks none. This table is the denominator: it includes the models that did not work, which a roster of supported models by construction cannot. It is not comparable cell by cell with the intervention table, which reports one specific 30-run sitting rather than a model’s best sitting, and the two can therefore differ by a cell or two for the same model.

Tool and reason	Count
compose_report : INVALID_TOOL_INPUT	6,243
compose_report : UNVERIFIABLE_EVIDENCE	3,464
tool : database-error	1,110
present_answer : INVALID_TOOL_INPUT	1,032
present_answer : ANSWER_NOT_A_DATA_READ	696
recommend_change : INVALID_TOOL_INPUT	613
recommend_change : RECOMMENDATION_SHAPE_MISMATCH	248
compare_plans : INVALID_TOOL_INPUT	152
compare_plans : UNVERIFIABLE_PLAN	139
All other reasons	311
Total	14,008

Table 6: Refused calls by tool and reason. `compose_report` alone accounts for 9,707 refusals, 69.3% of the total, split between calls whose *shape* did not validate and calls whose *citations* did not resolve. The single tool that turns completed work into a delivered answer is where two thirds of all refusals occur.

6.2 Displaced fields

Capturing the arguments of refused calls on two models over the two weakest surfaces produced 17 refusals in five runs per cell. Nine returned the same three-field refusal message. Four of those nine, all from one model, were the same displaced shape, differing only in the correlation id they cited:

```
{"change": "CREATE INDEX idx_employee_emp_no ON employee(emp_no);",
 "reason": "Creating an index on the employee table will improve the query performance ...",
 "evidence": {"correlationId": "772210b6-dcd8-4423-ae2b-21fd49a4b600", "source": "artifact"}}
```

The declared schema is

```
{change: "index"|"rewrite", statement: string, rationale: string, evidence:
      Evidence[]}
```

In those four every value is correct and three are displaced, each in its only plausible direction: the statement sits in the field named `change`, the rationale is called `reason`, and one evidence item arrived as itself rather than as a list of one. The model was told three fields were missing, about a call carrying all three values, four times. The remaining five of the nine came from a second model in three further shapes, `{column_name, index_name, table_name}`, `{column_name, index_name}` and `{sql}`, carrying between one and three of the required values against the same refusal text. A further five refusals carried a correct artifact id under a key one affix off. Across all released captures exactly one pair of calls is byte-identical, so the repetition here is of a shape rather than of a string.

6.3 The refusal that instructs the model to discard its answer

The clearest defect we found, and the one with the largest single-cell effect. On one 3B model's data-analysis cell, 0/5 with every loss recording a filed report and no presented answer while its other five surfaces locked, the captured arguments show a run that had both halves of the answer and filed one on the wrong tool:

```
present_answer <- {"artifact": "bfe7ca93-2bac-4df8-a091-a3d21bfa0454"}
compose_report <- {"claims": [...], "presentation": {"kind": "table"}}
```

The capture holds nine records for this cell in three identical groups: two calls to the answering tool carrying only the artifact and refused for a missing presentation, then one call to the reporting tool *carrying* a presentation and refused with the arguments object: `remove presentation`. Put together, any such pair is complete and correct.

The contradiction is structural and does not depend on what the model did next: the server demands `presentation` on one tool, forbids it on the sibling tool, and scores the run as having no answer when the field is absent, while the model has in fact composed one. `remove` is the correct word for a key that belongs nowhere and the wrong word for a key that is a field of a neighbouring tool, and the run is failed for the absence either way.

We state explicitly what the released capture does *not* establish. It carries no run identifier, no turn ordinal and no timestamp, so the order of these nine records cannot be recovered from it, and in file order the presentation-less calls precede the remove instruction rather than follow it. We therefore make no claim that the model deleted the field in response to being told to. The defect is the contradictory contract, which is visible without the ordering.

7 Interventions and measured effect

Five changes were made to the server on the strength of the above. None touches a model, a prompt, a temperature or a task.

#	Defect	Change
1	Text-channel calls invisible unless strictly parseable JSON keyed name	Recognise the call by its naming rather than by parsing the document
2	SQL placed in the enum field change	Read the kind from the statement; prose is still refused
3	rationale sent as reason ; evidence sent as one object or as a JSON string	Exactly-one-absent and exactly-one-surplus is an unambiguous rename; a lone object is a one-item list
4	A near-miss key named in the refusal for months and never applied	Apply the rename on flat two-field calls; leave nested calls to be told
5	remove <field> told models to discard a sibling tool’s field	remove reserved for keys that belong nowhere; a field owned by another tool is answered with where it goes

Table 7: The five server-side interventions. See §6.3 for the fifth.

Model	Before	After
granite4.2:3b	21/30	28/30
mistral-nemo:12b	18/30	26/30
llama3.1:8b	10/30	24/30
cogito:8b	0/30	21/30
glm4:latest	0/30	16/30
deepseek-r1:8b	18/28	24/28

Table 8: Cells open before the change, re-read after, same configuration and objectives. Scores are runs whose verdict was **answered**, resolved from the sweep logs’ run identifiers against the released dataset; the logs themselves record only process exit, which is not a pass. Two of **deepseek-r1:8b**’s thirty scheduled runs never started, so its denominator is 28 and two of its cells are short of the five runs a lock requires. The corpus-wide attrition is 89 scheduled runs that never started.

We report two qualifications that cut against these figures. First, the widened text-channel reader, intervention 1, measured in isolation on four cells produced +1, 0, 0, −1, which is inside variance. It is a correct fix serving a real 27-run population and it is not what moved the table; the argument-shape readers are. Second, part of the movement for three of these models is not attributable to code at all: their prior figures were unions of readings taken across days and settings, which §4.1 explains is not a valid row. Separating those two effects cleanly would require re-reading every prior row in one sitting, which we have not done. These are therefore field observations with known confounds, not an ablation.

8 Threats to validity

8.1 Unbounded serving context is a memory confound, and we hit it

With no context length configured, Ollama admits each model at its full advertised context. One 12B model, 7.1 GB on disk, was resident at 51 GB with a 262,144-token context on a 64 GB machine, and free memory fell to 6%.

Runs taken in that state are not measurements. A 3B model’s confirmation pass read its planning cell 0/5, all five runs hitting the 90-second turn limit having invoked no tool, a row that reads in any log exactly like a model that cannot plan. Capped at 32,768 the same model is 5.1 GB and the same cell reads 5/5.

Two properties make this dangerous for published work. System-wide free memory reads healthy, 76% in one sample, while a single model holds most of RAM, so the usual guard misses it. And the desktop application restarts its own server and reclaims the port, so a capped server fails to bind, logs an address-in-use error, and the uncapped server answers. That looks like success. The serving engine’s own process listing, with its context column, is the only check we found that does not lie.

All figures in Section 5 were taken before the cap was applied. The per-model and per-surface rates should therefore be read as a lower bound with unknown per-model bias, since large-context models were penalised more than small ones. Re-reading the corpus under a fixed cap is the first item of future work. We expect the taxonomy shape to survive it, because the class it should shrink is the clock class, and under our adopted ordering the clock class is not the largest. We note that this expectation cuts against us under the alternative ordering in Table 4, where shrinking the clock class moves runs into capability.

8.2 A related confound: swap

Separately, one sweep read a 24B model’s analysis cell at 0/5 with runs of 244 to 355 seconds against 41 to 48 seconds for the same cell earlier. Free memory read 53% throughout while swap sat at 19 GB of 20 GB with 1.45 million pageouts. Watchdogs that check free memory do not see this. What works is watching swap growth: a full but static swap file costs nothing, whereas swap being written while a model runs is the machine choosing between the model and everything else.

8.3 Single machine, single database, single harness

One machine, one schema, one agent implementation. The surface ordering and the taxonomy are properties of *this* tool contract. A contract with fewer structured artifacts per surface should show a smaller transport class, which is a testable prediction rather than a caveat.

8.4 Unequal sampling and unattributed runs

Run counts per model range from 3 to 629 and were allocated by operational interest rather than by design: models that looked close to completion were run more. Per-model rates therefore carry very unequal confidence and the corpus is not a randomised comparison. The taxonomy shares, computed over 2,100 agent-mode losses, are the more robust figure. Separately, 3,248 runs predate the ledger event that records the model and carry no attribution; they enter only the

whole-corpus figures. Two model tags in the corpus differ only in punctuation and may denote the same model, which would make the named count 39 rather than 40.

8.5 The run deadline is a hard boundary we did not move

One 8B model’s query-optimization cell passes at 443 and 444 seconds and fails at the 450-second run deadline. Raising the deadline would likely close it. We did not, because the deadline is part of the shipped product configuration, and a model that only passes outside it has not been shown to work for a user. This is a deliberate choice that costs us cells.

8.6 Uncertainty, and what it does not cover

These are uncontrolled field observations. Runs were not randomised, run counts per model were allocated by operational interest, and the corpus spans five code checkouts, so nothing here identifies a cause. Sampling uncertainty can still be quantified, and it should be, because the naive figure misleads in a specific direction.

Treating the 2,100 losses as independent gives narrow intervals: transport 36.2% [34.2, 38.3], clock 25.8% [24.0, 27.7], verification 20.7% [19.0, 22.5], capability 17.3% [15.7, 19.0], all Wilson 95%. They are independent only if failure mode is unrelated to model, which it plainly is not. Resampling the 33 contributing models rather than the runs, which is the correct unit, widens them to transport [22.0, 49.1], clock [10.1, 44.6], verification [8.5, 32.9] and capability [6.6, 31.4], and transport is the largest class in 74.5% of resamples rather than in all of them. Across the 22 models contributing at least twenty losses, transport is the largest class for 10 and capability the smallest for 13.

We therefore state the class ordering as a property of this corpus and not as a finding that generalises across models. What survives the clustered treatment is the engagement result: 1,590 of 2,100 losses, 75.7% [73.8, 77.5], came from runs that had invoked at least one tool; the majority holds in 99.7% of clustered resamples and in 15 of the 22 models with at least twenty losses. That is the claim the paper rests on, and it is the one that contradicts the capability-threshold framing.

The per-model before and after readings rest on thirty runs each, six cells of five rather than thirty independent trials. A sign test on six of six models improving gives $p = 0.031$. The cells were selected for re-reading because they were open, that is, because they had scored low, so regression to the mean predicts a positive mean delta even under a zero true effect; the two rows that began at 0/30 are the ones a floor protects from that. We report the six as field observations, not as an estimate of an effect size.

9 Discussion

Which mechanism? Our first reading of the transport class was informational: refusals failed because they stated the expectation and not what had arrived, so there was nothing in them to act on. The displaced-field data in §6 does not support that reading. In those runs the refusal named the missing fields explicitly and one model resent the same displaced shape four times while a second answered the same refusal text with four further shapes. A message containing the information needed to repair the call was not sufficient. That is what Gumaan’s counterfactuals predict: if most of the damage is the presence of the failed call’s surface form in the context, then improving the accompanying sentence addresses the smaller term. It is also consistent with what

actually moved our numbers. Interventions 2 to 5 did not produce better sentences; three of them changed what the server *accepts*, and the fifth replaced an instruction that was actively wrong. We read our data as evidence against a purely informational account.

Against the capability attribution. AgentBench attributes the open-versus-commercial gap to reasoning and instruction following. Our results do not refute that, and our own corpus contains models that never became usable. What they suggest is that any such attribution is only as good as the harness it was measured through, and that a benchmark which fixes its rejection messages and its argument parsing as part of the apparatus is measuring the pair rather than the model. Redesigning tool error feedback is already a standing recommendation [17], and ToolMisuseBench [15] already treats the feedback returned after a failed call as a declared condition; what we add is a measurement, in a deployed system, of what making that change is worth. We would extend the practice to the leaderboards that report per-model results, so that the rejection-message text and the parser’s tolerance are published as benchmark artefacts in their own right.

Practical advice. Record the arguments of refused calls somewhere, even when they must not enter the audit ledger: a shape that cannot be seen cannot be fixed. Separate refused-at-the-transport from tried-and-failed before reporting a zero. Check the serving engine’s resident context before trusting a timeout. And read a model’s cells in one sitting, because a union of readings is not a result.

10 Conclusion

The question of whether a model is big enough to drive an agent is the wrong first question. In 8,199 runs, under the classification we adopt and argue for, only one loss in six is a model that failed to engage; under the stricter alternative it is closer to one in four. Either way the majority of losses are models that engaged, established something true, and lost it to a field name, a brace, a channel, a clock, or a refusal that told them to throw the answer away.

That is an interface problem, and interface problems have the useful property that fixing them once helps every model at once. Five server-side changes, none model-specific, moved six models by 6 to 21 cells out of 30 on the same hardware, with the same prompts, on the same tasks. The distance left is measured in cells, not in parameters.

Author contributions (CRediT)

Cevheri Bozoğlan: conceptualisation, software, methodology, supervision, writing. **Yusuf Gündoğdu:** software, methodology, investigation, formal analysis, data curation, writing. **Abdullah Kaya:** investigation, data curation. **Koray Şirin:** software, validation.

Data and code availability

The application and its agent are at <https://github.com/libredb/libredb-studio> under the MIT licence. The measurement artefacts are *not* in that repository: the raw run ledgers are

written to a working directory the project gitignores, so they had never been published before this paper.

The complete measurement record is released as a dataset: 8,199 run records, 110,711 ledger events and 14,008 refusal records, with the exporter that produces them from the raw ledgers, a scorer that reproduces the per-model table, and a verification script that regenerates every figure in this paper and exits non-zero if any disagrees. It is published as a dataset [28], <https://doi.org/10.57967/hf/10485>, because the event stream alone exceeds the size a paper submission can carry. A reduced bundle accompanies this submission directly: the verification and scoring scripts, the captured arguments of refused calls, and the 160 per-cell sweep logs.

We ask readers to run the verifier before trusting a number here. Every figure in this paper was produced by it, and the classification in §4.4 agrees with the `loss_class` field of the released dataset on all 2,194 model-attributed losses, of which 2,100 are agent-mode.

Licence

This paper is released under CC BY 4.0. The dataset, the exporter, the scorer and the verifier are released under the MIT licence, as is the application.

Conflict of interest

All authors are contributors to LibreDB Studio, the software evaluated. No external funding supported this work.

References

- [1] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama. Is self-repair a silver bullet for code generation? *ICLR*, 2024. arXiv:2306.09896.
- [2] E. Gumaan. Feedback that backfires: why small language model agents repeat the call they just watched fail. arXiv:2608.23651, 2026.
- [3] X. Wang, Z. Wang, J. Liu, Y. Chen, L. Yuan, H. Peng, and H. Ji. MINT: evaluating LLMs in multi-turn interaction with tools and language feedback. *ICLR*, 2024. arXiv:2309.10691.
- [4] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou. Large language models cannot self-correct reasoning yet. *ICLR*, 2024. arXiv:2310.01798.
- [5] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen. CRITIC: large language models can self-correct with tool-interactive critiquing. *ICLR*, 2024. arXiv:2305.11738.
- [6] Z. Zhang, F. Zhao, R. Wang, et al. Robust tool use via Fission-GRPO: learning to recover from execution errors. arXiv:2601.15625, 2026.
- [7] S. V. Vuddanti, A. Shah, S. K. Chittiprolu, et al. PALADIN: self-correcting language model agents to cure tool-failure cases. arXiv:2509.25238, 2025.
- [8] R. Kadekodi, Z. Jin, K. Kamahori, et al. AgentFlux: decoupled fine-tuning & inference for on-device agentic systems. arXiv:2510.00229, 2025.

- [9] S. G. Patil, H. Mao, F. Yan, et al. The Berkeley Function Calling Leaderboard. *ICML*, PMLR 267:48371–48392, 2025.
- [10] Y. Qin, S. Liang, Y. Ye, et al. ToolLLM: facilitating large language models to master 16000+ real-world APIs. *ICLR*, 2024. arXiv:2307.16789.
- [11] M. Li, Y. Zhao, B. Yu, et al. API-Bank: a comprehensive benchmark for tool-augmented LLMs. *EMNLP*, 2023. arXiv:2304.08244.
- [12] J. Lu, T. Holleis, Y. Zhang, et al. ToolSandbox: a stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. arXiv:2408.04682, 2024.
- [13] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. τ -bench: a benchmark for tool-agent-user interaction in real-world domains. arXiv:2406.12045, 2024.
- [14] A. Sigdel and R. Baral. Schema first tool APIs for LLM agents: a controlled study of tool misuse, recovery, and budgeted performance. arXiv:2603.13404, 2026.
- [15] A. Sigdel and R. Baral. ToolMisuseBench: an offline deterministic benchmark for tool misuse and recovery in agentic systems. arXiv:2604.01508, 2026.
- [16] A. Mazumder and N. J. Lia. AgentCheck: a reproduce-intervene-mitigate workbench for LLM agents over MCP. arXiv:2607.11098, 2026.
- [17] Q. Xiong, Y. Huang, Z. Jiang, et al. Butterfly effects in toolchains: a comprehensive analysis of failed parameter filling in LLM tool-agent systems. arXiv:2507.15296, 2025.
- [18] H. Soni. ToolFailBench: diagnosing tool-use failures in LLM agents. arXiv:2607.04686, 2026.
- [19] X. Liu, H. Yu, H. Zhang, et al. AgentBench: evaluating LLMs as agents. *ICLR*, 2024. arXiv:2308.03688.
- [20] T. Yu, R. Zhang, K. Yang, et al. Spider: a large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL. *EMNLP*, 2018. arXiv:1809.08887.
- [21] J. Li, B. Hui, G. Qu, et al. Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs. *NeurIPS*, 2023. arXiv:2305.03111.
- [22] T. Yu, R. Zhang, M. Yasunaga, et al. SParC: cross-domain semantic parsing in context. *ACL*, 2019. arXiv:1906.02285.
- [23] T. Yu, R. Zhang, H. Y. Er, et al. CoSQL: a conversational text-to-SQL challenge towards cross-domain natural language interfaces to databases. *EMNLP*, 2019. arXiv:1909.05378.
- [24] F. Lei, J. Chen, Y. Ye, et al. Spider 2.0: evaluating language models on real-world enterprise text-to-SQL workflows. *ICLR*, 2025. arXiv:2411.07763.
- [25] X. Zhou, G. Li, Z. Sun, et al. D-Bot: database diagnosis system using large language models. *PVLDB*, 2024. arXiv:2312.01454.
- [26] H. Li, J. Zhang, H. Liu, et al. CodeS: towards building open-source language models for text-to-SQL. *SIGMOD*, 2024. arXiv:2402.16347.
- [27] P. Belcak, G. Heinrich, S. Diao, et al. Small language models are the future of agentic AI. arXiv:2506.02153, 2025.
- [28] C. Bozoğlan, Y. Gündoğdu, A. Kaya, and K. Şirin. Database agent runs: 8,199 tool-calling runs across 40 language models. Hugging Face, 2026. doi:10.57967/hf/10485.